INCIDENT REPORT

SOC Home Lab — Simulated Threat Detection Exercise

Report ID: IR-2026-003 **Classification:** CONFIDENTIAL — INTERNAL USE ONLY **Date:** May 7, 2026 **Analyst:** Daniel **Severity:** HIGH **MITRE ATT&CK:** T1543.003 — Create or Modify System Process: Windows Service **Status:** RESOLVED

---

1. EXECUTIVE SUMMARY

A simulated persistence attack was executed against a controlled Windows 10 endpoint. The attacker created a malicious Windows service called "WindowsHealthSvc" designed to execute automatically at every system boot with SYSTEM-level privileges — no user login required.

The attack was successfully detected by Sysmon (Event ID 13) and confirmed in Splunk within minutes of execution. An automated email alert fired to the analyst's inbox, identifying the affected machine and the malicious binary path. The full detection pipeline functioned as designed.

---

2. THE STORY — WHAT ACTUALLY HAPPENED

The Attacker's Mindset

Imagine an attacker who just broke into a company's network. Maybe someone clicked a phishing email. Maybe they exploited a vulnerability. Either way, the attacker now has a foothold on one Windows machine.

But here is the problem every attacker faces: **what happens when the machine reboots?**

If the malware only lives in memory, a single reboot wipes it out. All that work gone. The attacker loses their access and has to start over.

So attackers think ahead. They ask themselves: *"How do I make sure my malware survives that reboot?"*

This is called **persistence** — and creating a Windows service is one of the most powerful ways to achieve it.

Why Services Are The Attacker's Favourite Trick

A Windows service is a program that Windows runs automatically in the background, before any user even logs in, with the highest possible privileges on the machine. Every time the computer starts up, Windows reads a list of services and launches them all — things like your printer driver, your network adapter, your antivirus.

The attacker's goal is simple: **get their malware onto that list.**

Once they do, the rules change completely:

- Server reboots at 3am for updates — malware comes back automatically
- Nobody logs in for three days — malware is still running
- User changes their password — doesn't matter, the service doesn't need credentials
- Antivirus kills the running process — Windows restarts it for the attacker

That last point is the most dangerous. Windows becomes the attacker's accomplice, automatically resurrecting the malware every time something kills it.

Real World Examples

This is not theoretical. Real ransomware groups use this exact technique:

- **WannaCry** — created services to spread across networks
- **Emotet** — registered services with names mimicking legitimate Windows components
- **Cobalt Strike beacons** — one of the most used attacker tools in the world, uses service creation as a primary persistence method

---

3. ENVIRONMENT

Component	Details
Endpoint	Windows 10 VM (WindowsVM10)
Detection Tool	Sysmon v15 + Splunk Enterprise
Sysmon Event	Event ID 13 — Registry Value Set

Registry Path	HKLM\SYSTEM\CurrentControlSet\Services
Analyst Machine	Host OS running VirtualBox

---

4. ATTACK SIMULATION

What Was Executed

The following PowerShell command was run as Administrator on the Windows 10 VM:

```
New-Service -Name "WindowsHealthSvc" `
    -BinaryPathName "C:\Windows\System32\cmd.exe /c whoami" `
    -DisplayName "Windows Health Service" `
    -StartupType Automatic `
    -Description "Monitors Windows health and performance"
```

Breaking Down The Attack Command

Every part of this command mirrors what real malware does:

Name: "WindowsHealthSvc" The attacker picks a name that sounds like a real Windows component. This is called masquerading. A junior analyst glancing at a services list might see "Windows Health Service" and assume it is legitimate. This is exactly how APT groups blend in.

BinaryPathName: "C:\Windows\System32\cmd.exe /c whoami" This is the binary path — the actual program the service will run. In our simulation, it runs `whoami` which is harmless. In a real attack, this would point to a reverse shell, a credential dumper, or ransomware. This field is the smoking gun that our detection rule targets.

StartupType: Automatic This tells Windows to start the service every single time the computer boots — with no user interaction required.

Description: "Monitors Windows health and performance" A convincing fake description. Real malware uses legitimate-sounding descriptions to avoid suspicion during manual investigation.

Why The Payload Does Not Matter For Detection

The `whoami` command is harmless. But our detection does not care about the payload. What we are detecting is the **behaviour** — a shell process (PowerShell or cmd.exe) creating a service with a suspicious binary path. That behaviour is identical whether the payload runs

`whoami` or deploys ransomware.

This is the key insight of behavioural detection: **detect the action, not the specific tool.**

---

5. WHAT SYSMON CAPTURED

When the PowerShell command ran, Sysmon generated **Event ID 13 — Registry Value Set**.

This event fires whenever a process writes a value to the Windows registry. Since all services are stored in the registry under `HKLM\SYSTEM\CurrentControlSet\Services`, creating a service always generates this event.

Evidence Captured In Splunk

Field	Value	Significance
EventCode	13	Registry modification — service being written
Image	C:\Windows\system32\services.exe	Windows Service Control Manager handling the write
Details	C:\Windows\System32\cmd.exe /c whoami	The malicious binary path — attacker's payload location
Details	Windows Health Service	Fake display name designed to look legitimate
Details	LocalSystem	Service will run with SYSTEM privileges
User	NT AUTHORITY\SYSTEM	Highest privilege account on the machine
ComputerName	WindowsVM10	Affected endpoint identified

Why services.exe Appears As The Image

A common question: why does the Image field show `services.exe` and not `powershell.exe`?

When PowerShell runs `New-Service`, it sends the request to the Windows Service Control Manager (`services.exe`). The SCM is the official Windows component responsible for

writing service entries to the registry. So `services.exe` appears as the process that physically wrote to the registry — but it was acting on behalf of PowerShell.

This is normal Windows behaviour and does not reduce the detection value. The malicious binary path in the Details field is still there, still suspicious, and still caught by our rule.

---

6. DETECTION RULE

The Logic

Our detection targets services where the binary path contains executables that have no business running as a service:

```
index=endpoint EventCode=13 "CurrentControlSet\\Services"
(Details="*cmd.exe*" OR Details="*powershell.exe*" OR
Details="*C:\\Users\\Public*" OR Details="*C:\\Temp*")
| eval AttackType="Suspicious Service Created"
| stats count by ComputerName, Details, AttackType
| sort -count
```

Why Each Filter Was Chosen

cmd.exe and powershell.exe in binary path No legitimate software installs a Windows service that points to a command prompt. Legitimate services point to dedicated executables in Program Files. The moment you see cmd.exe or PowerShell in a service's binary path, that is an attacker.

C:\Users\Public and C:\Temp Legitimate software installs to Program Files, which requires administrator rights. Malware often drops itself in Public or Temp folders because any user can write there without elevated privileges. A service binary in these locations is a strong indicator of compromise.

Why LocalSystem Was Excluded During investigation, adding `LocalSystem` as a filter increased results from 3 to 9. The extra 6 results were legitimate Windows services — false positives. LocalSystem is used by nearly all Windows services, both legitimate and malicious. Using it as a standalone filter creates noise. The binary path filters alone are specific enough to isolate the attack.

The eval AttackType Field In a real SOC with dozens of detection rules firing simultaneously, analysts need to know immediately what they are looking at. Labelling each result with `AttackType="Suspicious Service Created"` allows analysts to triage faster and maps cleanly to dashboards and ticketing systems.

Detection Results

The rule returned exactly **3 events** — all corresponding to the simulated attack. Zero false positives. High fidelity, low noise.

---

7. AUTOMATED ALERT

An alert was configured in Splunk to run the detection rule on a schedule and fire an email notification when results exceed zero.

Alert Configuration

Setting	Value
Alert Name	Suspicious Service Creation Detected
Schedule	Every hour
Time Window	Last 24 hours
Trigger Condition	Number of results > 0
Action	Send email notification

Email Alert Received

The email alert arrived in the analyst inbox and contained:

- **Subject:** ALERT — Suspicious Service Creation on WindowsVM10
- **Machine Name:** WindowsVM10 — immediately identifies the affected endpoint
- **Binary Path:** C:\Windows\System32\cmd.exe /c whoami — the evidence
- **Attack Type:** Suspicious Service Created — pre-labelled for triage
- **Direct Splunk Link** — one click to begin investigation

In a real SOC, this email would become a ticket. The analyst would click the link, pull up the full event timeline, cross-reference with other activity on that machine, and begin the response process.

---

8. INVESTIGATION STEPS PERFORMED

1. Ran the attack simulation on the Windows 10 VM
2. Confirmed service creation via `Get-Service -Name "WindowsHealthSvc"`
3. Confirmed service registration via `sc.exe qc WindowsHealthSvc`
4. Searched Splunk for Event ID 13 targeting the service registry path
5. Identified the malicious binary path in the Details field
6. Refined detection rule — removed LocalSystem filter after confirming it introduced false positives
7. Added eval labelling and stats grouping for production-grade output
8. Saved detection as a scheduled alert with email notification
9. Confirmed email alert received with correct machine name and evidence
10. Cleaned up simulation artefacts

---

9. CLEANUP

After confirming detection, the simulated malicious service was removed:

```
Stop-Service -Name "WindowsHealthSvc" -Force
Remove-Service -Name "WindowsHealthSvc"
```

This mirrors real incident response. After an attacker's persistence mechanism is identified and documented, the response team removes it and validates the system is clean.

---

10. MITRE ATT&CK MAPPING

Field	Value
Tactic	Persistence (TA0003)
Technique	Create or Modify System Process (T1543)
Sub-Technique	Windows Service (T1543.003)
Data Source	Windows Registry, Sysmon Event ID 13

Detection	Registry modification to CurrentControlSet\Services with suspicious binary path

Real-world threat groups documented by MITRE as using this technique include APT groups and ransomware operators. This technique appears in investigations involving WannaCry, Emotet, and Cobalt Strike-based intrusions.

---

11. LESSONS LEARNED

Lesson 1 — Precision Over Volume

When testing the detection rule, adding LocalSystem as a filter increased results from 3 to 9. Those extra 6 were legitimate Windows services — false positives. In a real SOC, false positives waste analyst time and create alert fatigue, which causes real attacks to be missed.

The lesson: a detection rule that returns 3 real attacks and 0 false positives is more valuable than one that returns 3 real attacks and 6 false positives. Always tune for precision.

Lesson 2 — The Binary Path Is The Smoking Gun

The most important field for detecting malicious service creation is the binary path in the Details column. No legitimate software installs a service pointing to cmd.exe, PowerShell, or a temporary folder. This single indicator is enough to separate attacker activity from normal system behaviour with near-zero false positives.

Lesson 3 — Understand Your Log Sources

Services.exe appeared as the Image rather than PowerShell. Understanding why — that the Service Control Manager acts as the intermediary for registry writes — prevented a false conclusion that the detection had missed the attack. Knowing the difference between who initiated an action and who physically executed it is critical for accurate investigation.

Lesson 4 — Services Survive Reboots Without User Interaction

This is what makes service creation more dangerous than scheduled tasks or registry run keys. Both of those require a user to log in before the malware executes. A service runs the moment the machine boots, making it the preferred persistence mechanism for attackers targeting servers where nobody logs in interactively.

---

12. RECOMMENDED RESPONSE ACTIONS (Production Environment)

If this alert fired on a real corporate endpoint:

1. **Isolate the endpoint immediately** — disconnect from network to prevent lateral movement
2. **Preserve evidence** — capture memory dump and disk image before making changes
3. **Remove the malicious service** — Stop-Service followed by Remove-Service or sc delete
4. **Hunt for additional persistence** — check registry run keys, scheduled tasks, startup folders
5. **Review authentication logs** — check for unusual login activity before and after service creation
6. **Check for lateral movement** — review network connections from the affected machine
7. **Force credential reset** — assume credentials on the machine are compromised
8. **Escalate to incident response team** — service creation with SYSTEM privileges indicates a serious breach

---

13. EVIDENCE ARTEFACTS

Item	Description
PowerShell screenshot	Attack command execution showing service created
Splunk Event ID 13 screenshot	Raw events showing binary path captured
Splunk detection rule screenshot	Final tuned query returning 3 clean results
Email alert screenshot	Automated notification showing machine name and binary path

---

14. PORTFOLIO STATEMENT

“I simulated and detected a Windows Service persistence attack mapped to MITRE ATT&CK T1543.003. Using Sysmon Event ID 13, I captured registry modifications to the Windows service registry path and built a Splunk detection rule that identifies services with suspicious binary paths — specifically targeting cmd.exe, PowerShell, and writable directory locations where malware commonly drops itself. I tuned the rule to eliminate false positives after discovering that broader filters like LocalSystem introduced noise without improving accuracy. The final rule returned zero false positives against three simulated attack events. I configured automated email alerting that fires within one hour of a suspicious service creation, delivering the affected machine name and binary path evidence directly to the analyst inbox.”

Report prepared by: Daniel | SOC Home Lab Portfolio Detection Technique: Behavioural — Registry Modification Analysis For portfolio and interview demonstration purposes